



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»»

Лабораторная работа № 4 по дисциплине «Анализ алгоритмов»

Тема Программирование параллельных потоков

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Содержание

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Потоки	4
1.2 Связность графа	4
1.3 Задание по варианту	4
2 Конструкторская часть	5
2.1 Описание алгоритмов	5
3 Технологическая часть	6
3.1 Реализация алгоритмов	6
3.2 Тестирование	7
4 Исследовательская часть	8
4.1 Замеры процессорного времени	8
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

ВВЕДЕНИЕ

Цель работы – разработка и сравнительный анализ последовательного и параллельного алгоритмов.

Задачи, необходимые для достижения поставленной цели:

- 1) описать последовательный алгоритм решения задачи по варианту и его параллельную версию;
- 2) реализовать оба алгоритма;
- 3) выполнить сравнительный анализ зависимостей времени решения задач от размерности входа для реализации последовательного алгоритма и для реализации модифицированного алгоритма, запущенной с единственным вспомогательным потоком;
- 4) выполнить сравнительный анализ зависимостей времени решения задач от размерности входа для реализации модифицированного алгоритма при k вспомогательных потоках;
- 5) сформулировать рекомендацию о выборе k для решения задачи на ЭВМ архитектуры.

1 Аналитическая часть

1.1 Потоки

Поток – наименьшая единица работы, которая может быть назначена операционной системе для выполнения. Поток выполняется внутри процесса, и несколько потоков могут существовать внутри одного процесса, работая параллельно [?].

Нативный поток – поток выполнения, которым управляет непосредственно операционная система и которые напрямую взаимодействуют с ядром операционной системы.

Зеленый поток – поток, который управляется средой выполнения (например, виртуальной машиной).

1.2 Связность графа

Связность графа – качество графа, отображающее наличие путей между его вершинами и их количество [3].

Компонента связности – максимальный подграф неориентированного графа, вершины которого соединены между собой. Компонента сильной связности – максимальный подграф ориентированного графа, каждая пара вершин которого достижима между собой. Компонента слабой связности – максимальный подграф ориентированного графа, каждая пара вершин которого достижима между собой, если этот граф рассматривается как неориентированный.

1.3 Задание по варианту

Проверка связности графа, поиск всех компонент связности (ответ – по списку вершин на каждую компоненту связности).

Дан граф в формате graphviz. Входные данные считываются из файла. Граф не полностью связный, количество вершин не менее 3000, из/в вершину ведут, вероятнее всего, от 0 до 15 дуг.

Вывод

В данном разделе мной был проведен анализ теоретических материалов, объясняющих принцип работы алгоритмов.

2 Конструкторская часть

2.1 Описание алгоритмов

Схемы последовательного и распараллеленного алгоритмов представлены на рис. 2.1 и 2.2 соответственно.

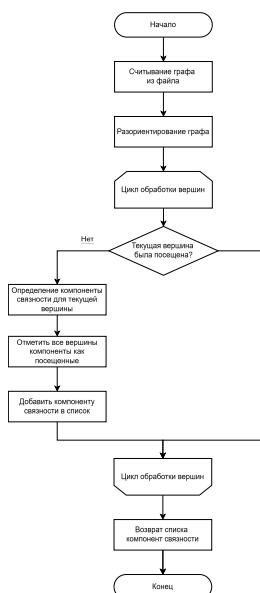


Рисунок 2.1 — Схема последовательного алгоритма

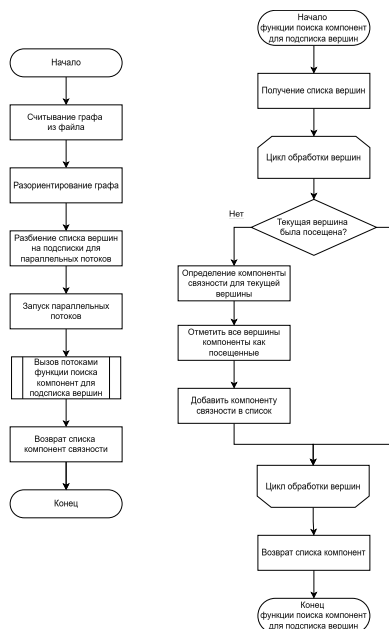


Рисунок 2.2 — Схема параллельного алгоритма

Вывод

В данном разделе мной были построены схемы последовательного и параллельного алгоритмов.

3 Технологическая часть

3.1 Реализация алгоритмов

Для реализации алгоритмов использовался язык программирования C++, так как он поддерживает статическую типизацию, имеет возможность отключения сборщика мусора и поддерживает нативную многопоточность (библиотека `threads` [1, с. 1217]). Отладка происходила в среде разработки «Visual Studio 2022».

```
1 std::vector<std::unordered_set<std::string>> graphviz_graph::find_components(){
2     std::unordered_map<std::string, std::vector<std::string>> graph1 = undirect();
3     std::vector<std::unordered_set<std::string>> components;
4     std::unordered_set<std::string> component;
5     std::string node;
6     std::unordered_set<std::string> visited;
7     for (const auto& kv: graph){
8         node = kv.first;
9         if (visited.find(node) == visited.end()){
10             component = bfs(node, graph1);
11             visited.insert(component.begin(), component.end());
12             components.push_back(component);
13         }
14     }
15     return components;
16 }
```

Листинг 1 – Последовательный алгоритм.

```
1 std::vector<std::unordered_set<std::string>> graphviz_graph::find_components_parallel(int
   thread_cnt){
2     std::unordered_map<std::string, std::vector<std::string>> graph1 = undirect();
3     std::vector<std::unordered_set<std::string>> components;
4     std::vector<std::string> nodes;
5     for (const auto& kv: graph)
6         nodes.push_back(kv.first);
7     std::unordered_set<std::string> visited;
8     std::mutex c_mutex, v_mutex;
9     std::vector<std::thread> threads(thread_cnt);
10    int nodes_cnt = nodes.size(), n, step, k = 0, kf;
11    if (nodes_cnt <= thread_cnt) {
12        thread_cnt = nodes_cnt;
13        n = 1;
14        step = 1;
15    }
16    else {
17        n = nodes_cnt / thread_cnt + ((nodes_cnt % thread_cnt > 0) ? 1 : 0);
18        step = std::min(n, nodes_cnt);
19    }
20    for (int i = 0; i < thread_cnt; i++) {
21        kf = std::min(k + step, nodes_cnt);
22        threads[i] = std::thread(task, this, k, kf, std::ref(nodes), std::ref(components),
23                                std::ref(c_mutex), std::ref(visited), std::ref(v_mutex), std::ref(graph1));
24        k += step;
25    }
26    for (int i = 0; i < thread_cnt; i++){
27        if (threads[i].joinable())
28            threads[i].join();
29    }
```

```

28     }
29     return components;
30 }
31
32 void graphviz_graph::task(const int st, const int fn, const std::vector<std::string>& nodes,
    std::vector<std::unordered_set<std::string>>& components, std::mutex& cm,
    std::unordered_set<std::string>& visited, std::mutex& vm, const
    std::unordered_map<std::string, std::vector<std::string>>& graph){
33     std::unordered_set<std::string> component;
34     std::unordered_set<std::string> local_visited;
35     bool v_node;
36     for (int i = st; i < fn; i++){
37         if (!local_visited.insert(nodes[i]).second)
38             continue;
39         {
40             std::lock_guard<std::mutex> lk(vm);
41             if (!visited.insert(nodes[i]).second)
42                 continue;
43         }
44         component = bfs(nodes[i], graph);
45         local_visited.insert(component.begin(), component.end());
46         {
47             std::lock_guard<std::mutex> lk(vm);
48             visited.insert(component.begin(), component.end());
49         }
50         {
51             std::lock_guard<std::mutex> lk(cm);
52             components.push_back(component);
53         }
54     }
55 }

```

Листинг 2 – Параллельный алгоритм.

3.2 Тестирование

Таблица 3.1 — Модульные тесты для алгоритмов.

Алгоритм	Вход	Выход
Последовательный	testgraph.gv	{ n26 n43 n0 n39 n44 n18 n32 n35 n31 n38 n36 n40 n33 n48 n27 n19 n37 n2 n46 n12 n14 n16 }, { n5 n17 n45 n30 n20 }, { n21 n1 n25 }, { n34 n24 n7 }, { n9 n6 }, { n4 n29 }, { n15 n49 }
Параллельный	testgraph.gv	{ n26 n43 n0 n39 n44 n18 n32 n35 n31 n38 n36 n40 n33 n48 n27 n19 n37 n2 n46 n12 n14 n16 }, { n5 n17 n45 n30 n20 }, { n25 n1 n21 }, { n34 n24 n7 }, { n4 n29 }, { n9 n6 }, { n15 n49 }

Вывод

В данном разделе мной были реализованы последовательный и параллельный алгоритмы и проведено их успешное тестирование.

4 Исследовательская часть

4.1 Замеры процессорного времени

Замеры проводились на ноутбуке RedmiBook Pro 14 2022:

- процессор Intel i7-12650H (архитектура x86-64, 10 ядер по 2.3 ГГц, 16 логических ядер);
- оперативная память 16 Гб (DDR4, 5200 МГц);
- операционная система Windows 11 Home 25H2.

Для замеров использовалась функция `clock()` из библиотеки `ctime` [1, с. 288].

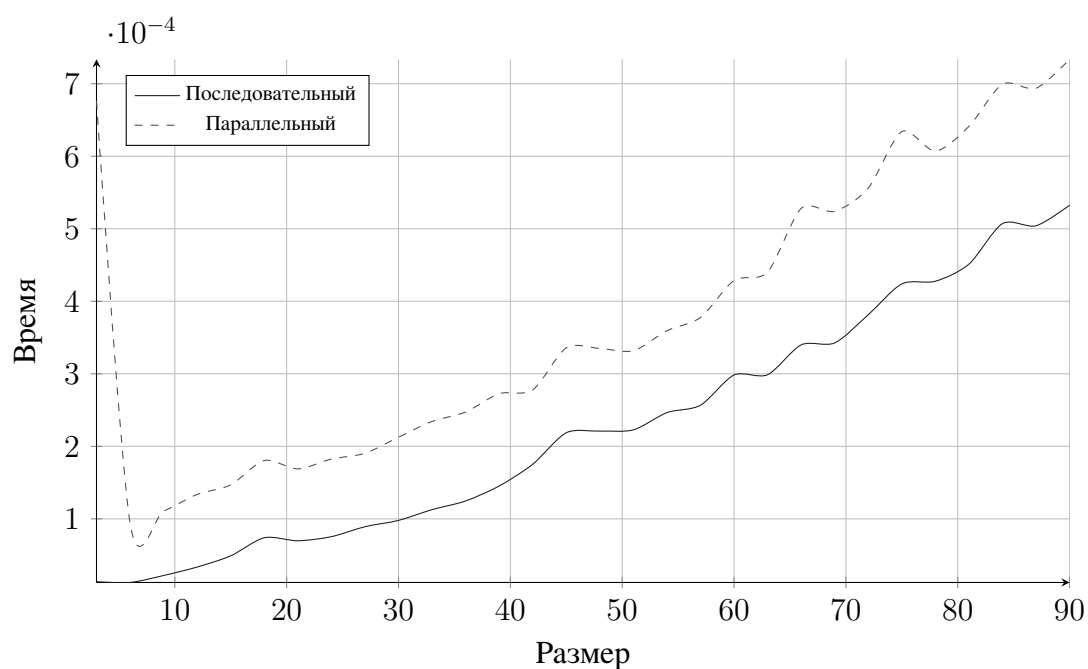


Рисунок 4.1 — График зависимости процессорного времени от размерности входа при $k=1$.

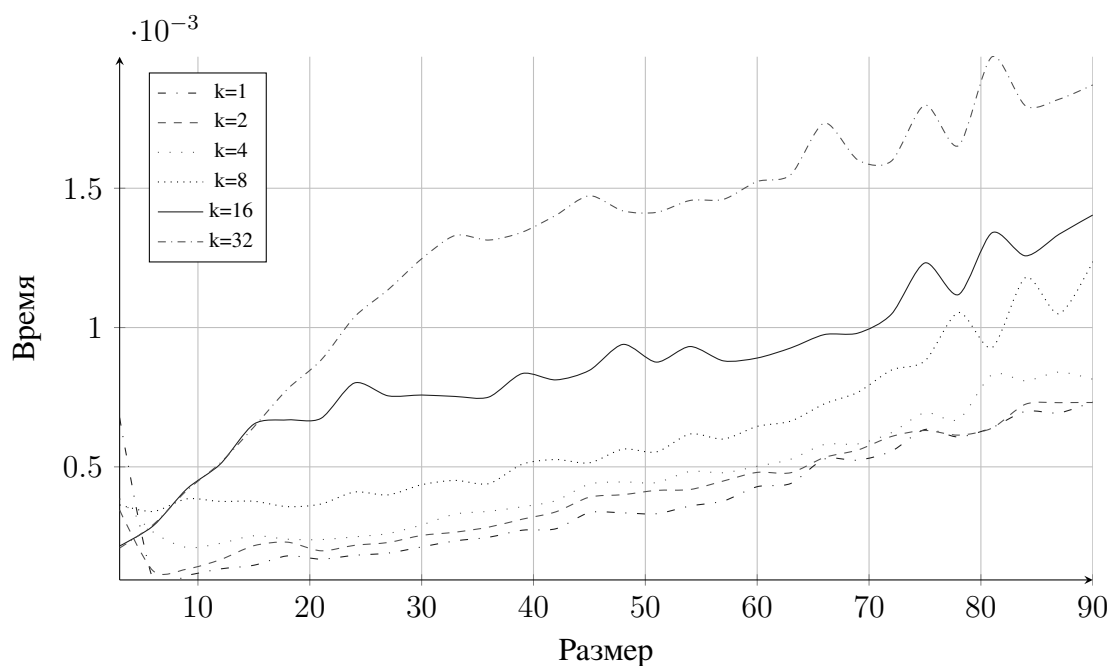


Рисунок 4.2 — График зависимости процессорного времени от размерности входа при различных $k = \{1, 2, \dots, 32\}$.

Вывод

В данном разделе были проведены замеры реального времени при одном рабочем потоке для каждой реализации из предыдущего раздела. Также мной было проведен сравнительный анализ зависимости времени выполнения параллельной реализации при разном количестве потоков. Самое оптимальное количество потоков – 16.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были описаны последовательный и параллельный алгоритмы поиска компонент связности и проведен их сравнительный анализ. Из анализа следует, что для решения поставленной задачи наиболее оптимальным количеством рабочих потоков является 16.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. C17 standard (ISO/IEC 9899:2018) [стр. 288]. [Электронный ресурс] Режим доступа: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (дата обращения: 29.09.2025).
2. Теория графов. Часть 1: учебное пособие / Ю.Б.Буркатовская; Томский политехнический университет. – Томск: Изд-во Томского политехнического университета, 2014. – 200 с. – Текст: непосредственный.
3. Уильямс Э. С++. Практика многопоточного программирования. / Уильямс Э. СПб, 2020. – 640с. – Текст: непосредственный.